

UNIVERSIDAD AUTONOMA DE AGUASCALIENTES

**INGENIERIA EN COMPUTACION
INTELIGENTE**

**ADMINISTRACION DE SOFTWARE Y
PROYECTOS**



PLANIFICACION SPRINT 3

Arrioja Pizaña Cesar Emmanuel

Cruz Mora Ángel

Semestre VIII

Interfaz Móvil y Vista de Ruta para el Repartidor

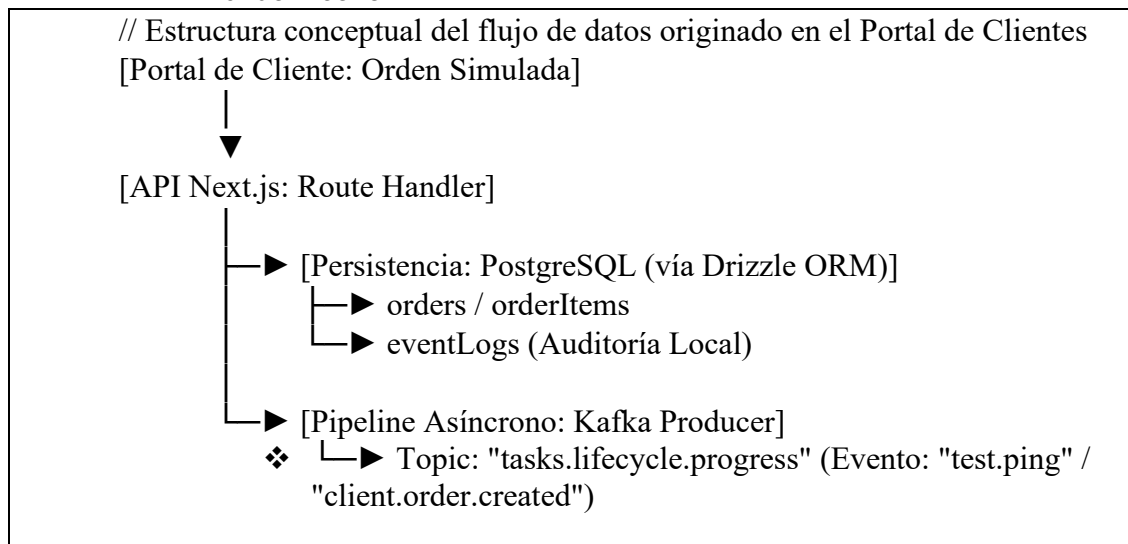
Este sprint se enfocó de manera integral en el brazo operativo en campo: los conductores o repartidores (*Drivers*). Con el objetivo de eliminar el uso de papel en los camiones de distribución, el desarrollo se centró en una interfaz optimizada para dispositivos móviles ("*Mobile-First*"). A nivel de código, se diseñó la lógica para capturar las órdenes que ya han sido procesadas por la oficina y presentarlas al operador de forma limpia y directa. El avance se tradujo en una vista funcional donde el conductor puede revisar su itinerario del día y los detalles específicos de los activos que debe trasladar, eliminando las pérdidas o daños asociados a los formatos físicos.

Se deberá transicionar el Portal de Cliente de un entorno puramente estático/local a un ecosistema conectado mediante el aprovisionamiento de la base de datos relacional (PostgreSQL + Drizzle ORM) y el diseño de los primeros productores de mensajería asíncrona con Apache Kafka para digerir el ciclo de vida de los contratos.

LOGROS ALCANZADOS:

Logro A: Modelado de Datos de la Orden (Drizzle ORM & Postgres)

- Se estructuró el esquema relacional que da soporte al comportamiento polimórfico del inventario del portal de clientes. Dado que el negocio maneja productos de catálogo para fiestas (`partyProducts`) y estructuras a gran escala (`eventProducts`), el modelo de base de datos ahora soporta el almacenamiento unificado en `orderItems`.



Logro B: Arquitectura del Productor de Eventos (Event-Driven Gateway)

Para dar salida a las órdenes que el cliente genera en el portal, se construyó el core del cliente productor de Kafka (produceEvent). Esto asegura que cuando un cliente pulsa "Confirmar Orden de Simulación", el hilo de ejecución principal de la aplicación web quede libre de inmediato. Next.js procesa la inserción rápida en base de datos y delega el análisis de la orden al clúster de Kafka.

Logro C: Automatización del Estado 'PENDING' (Pool Target)

Toda orden simulada e ingresada desde el Portal de Cliente se inicializa estrictamente con el estatus PENDING. Este estado es el disparador crítico que posteriormente permitirá a los miembros de la oficina visualizar los contratos entrantes y balancear la carga de trabajo entre los transportistas (*Drivers*).

Con las órdenes simuladas guardadas con éxito en estado PENDING y transmitidas mediante eventos de Kafka en este Sprint 3, la infraestructura queda lista para que en el **Sprint 4** se implemente el **Consumidor General con Server-Sent Events (SSE)**. Esto permitirá que la interfaz de la administración web (Oficina) "escuche" en tiempo real las órdenes que los clientes simulan en el portal e inicien de inmediato el proceso de asignación a los camiones repartidores.

EVIDENCIA:

